PLAYCARYS · PLATFORM ECONOMICS

Revenue Model

Five streams, built in the order that makes each one worth building. Audited against the tables already sitting in live_database.json.

14 AUGUST 2026 · MEASURED AGAINST THE WORKING TREE · 13 users / 15 games / 96 matches / $0 earned

01 Where the money is today

Nowhere. The platform has a complete progression economy and a complete shop catalogue, and no path between them or out of them. Six monetisation tables exist in the database with **zero lines of code** reading or writing them; they survive on disk only because the backend passes unknown keys straight through to the file.

6	0	$64.86	0
MONETISATION TABLES	REFERENCED IN CODE	TOTAL SEEDED REVENUE	PAID TO PUBLISHERS

What is actually in them

Table	Rows	Contents	Verdict
`cosmetic_items`	30	Catalogue. Six categories of five, four rarity bands, $0.99–$4.99. Each carries a CSS gradient or emoji as its preview.	**Keep as-is**
`user_cosmetics`	8	Who owns what, and which item they are wearing.	**Needs a key**
`subscription_plans`	2	Pro $4.99/mo and Pro+ $9.99/mo, each with a feature list.	**Rewrite the perks**
`subscriptions`	2	Who is on which tier, with start and expiry dates.	**Keep as-is**
`transactions`	16	$64.86 across three types: subscription, cosmetic_purchase, tournament_entry.	**Needs real states**
`ad_placements`	5	Five zones, CPM $2–$12, impressions and earnings, all targeting the free tier.	**Shelve for now**

The data quality is good

A full referential-integrity check returns **zero orphaned rows**. Every `user_id` resolves to a real account, every `item_id` to a real cosmetic, every subscription tier to a real plan. The transaction rows even reconcile against `user_cosmetics` by date *and* by price. The catalogue-and-junction pair is the cleanest relational modelling anywhere in the file. This is a good skeleton; it is missing muscles, not bones.

What is missing

- **Two currencies that never meet.** Everything here is priced in US dollars. The platform already runs on Carys Points, earned by playing, with seven tiers from Iron to Master. There is no way to buy Carys and no way to spend them.
- **Publishers earn nothing.** Not one row anywhere pays a studio. This is the largest hole, because studios uploading games is the entire premise of the platform.
- **Tournament entry fees are fiction.** Two transactions record them, but a tournament row has no entry-fee field and states its prize as the *string* "$2500". Money leaves a pool that nothing fills.

- **Every transaction is "completed".** No pending, no failed, no refunded. Real payments fail constantly, and a schema with one state cannot record that.
- **Prices are floating-point numbers.** 4.99 has no exact binary representation. Money must be stored as integer minor units or as SQL DECIMAL.

> **YOU ALREADY BUILT A ROYALTY ENGINE**
> Economy.awardPostMatchProgression in js/utils.js quietly credits a publisher **15 Carys per session played** into publisher_profiles.revenue_pcp. The live data holds 45,000 and 12,500 in those fields, so it has been running for months. Two problems: the field is not declared in the backend PublisherProfile type, and Carys converts to nothing — so it is a counter measuring an imaginary quantity. Stream 01 turns it into money.

02 The five streams, in build order

The order is the argument. Each stream makes the next one worth building, and the first one is what turns a games website into a platform.

01 Publisher revenue share — 70 / 30

Today a studio uploads a game, passes review, gets played, and receives nothing it can spend. There is no reason for anyone to publish here. Nothing else on this list matters until that is false.

The publisher keeps **70%** of everything attributable to their game: its share of tournament rake, its ad impressions, its paid unlocks. The platform keeps 30%. Needs a payouts table, a real money balance on the publisher profile, and a minimum payout threshold so you are not wiring out $0.40.

`UNLOCKS: A SUPPLY SIDE. WITHOUT IT, STREAMS 02-05 HAVE NOTHING TO SELL.`

02 Make Carys a two-way currency

Carys only flows inward. Players accumulate points that buy nothing — your own account holds 149 of them and they do not mean anything. Open the two missing valves.

Outward: every cosmetic gains a price_carys beside its price_usd. Thirty items become spendable on the day you add the column. **Inward:** players buy Carys with money, at a rate where cash is the fast path and play is the free one.

`THE HINGE. CONNECTS THE ECONOMY YOU BUILT TO THE SHOP THAT WAS SEEDED.`

03 Subscriptions — keep the tiers, rewrite the perks

$4.99 and $9.99 are sensible price points, and "no ads" is a real benefit. But "Full match history" and "Profile customization" are weak: users assume those are free, so charging for them reads as a penalty rather than a reward.

Replace the weakest perk on each tier with a **monthly Carys stipend**. That is the retention mechanic — the subscription renews because the currency renews. Add a reduced platform cut for subscribers who organise tournaments.

`DEPENDS ON 02. A STIPEND IS WORTHLESS UNTIL CARYS BUYS SOMETHING.`

04 Tournaments — entry fee and rake

Replace prize:"$2500" with real fields: entry_fee, currency, rake_pct, and a prize_distribution table so the 60/25/10 split is data rather than a hardcoded formula. Prize pool = entries x fee x (1 - rake). Platform takes 10-15%.

Allow entry in Carys *or* cash. Carys entries cost you nothing and drive engagement; cash entries are margin. This also fixes a live bug — buildPrizeBreakdown once printed the whole pool for all three tiers.

`NEEDS THE SCHEMA CHANGES IN SECTION 03.`

05 Advertising — build it last

The seeded table is unintentionally honest: $715 earned across 185,000 impressions. That is the true shape of ad revenue — it needs enormous traffic before it buys a coffee, and it is the stream that costs the most goodwill.

Keep the table. Wire it when the traffic exists. Its real job in the meantime is to be the thing that Pro removes.

`ITS target_tier:"free" FIELD IS ALREADY CORRECT. GOOD INSTINCT.`

DECIDE THIS BEFORE BUILDING ANY OF IT

Carys sits at the centre of all five streams, so its exchange rate is the most consequential number in the platform. Too generous and nobody buys anything; too mean and playing feels pointless. Hill Climbing pays score^0.75/12 and a chess win pays 150 — work out what **one hour of play** is worth in Carys first, then price the shop against that hour. Not the other way round.

03 Schema changes the model requires

These are the columns without which none of the five streams can be recorded correctly. Everything here assumes the move to a real database described in the companion scalability report.

Table	Change	Which stream needs it
`publisher_profiles`	Add balance_minor BIGINT and lifetime_earned_minor BIGINT. Keep revenue_pcp as the engagement counter it already is, and declare it in the TypeScript type where it is currently missing.	**01**
`payouts`	New table: publisher_id, period, gross_minor, platform_cut_minor, net_minor, status, paid_at. A payout is a historical fact and must be append-only.	**01**
`cosmetic_items`	price becomes price_minor INT; add price_carys INT NULL. A null means the item is cash-only.	**02**
`user_cosmetics`	PRIMARY KEY (user_id, item_id), plus a partial unique index so only one item per category can be equipped at a time.	**02**
`carys_ledger`	New table, double-entry: user_id, delta, reason, ref_type, ref_id, balance_after. Once Carys has cash value, player_profiles.carys_points alone is no longer defensible — you need to be able to prove where every point came from.	**02**
`subscriptions`	Add stipend_carys INT and last_stipend_at. Without them a renewal cannot be told apart from a first payment.	**03**
`tournaments`	Replace prize VARCHAR with entry_fee_minor, entry_currency, rake_pct, prize_pool_minor. Add a prize_distribution child table.	**04**
`transactions`	amount becomes amount_minor INT. status gains pending / failed / refunded / disputed. Add provider, provider_ref and an idempotency_key with a unique constraint, so a retried payment cannot charge twice.	**all**

THE ONE NON-NEGOTIABLE ORDERING RULE

Do not put real payments into a JSON file that two browser tabs can overwrite. Every data-loss bug fought so far has cost match history. The same bug on the transactions table costs someone their money — and unlike a lost match, that cannot be regenerated by playing again. **Postgres before payments.**

Sequence

#	Do this	Why in this position
1	**Password reset flow**	Still missing. With the device-handoff path, a stolen login means permanent lockout. The only open item that harms a real person, so it precedes all revenue work.
2	**Carys can be spent (stream 02, outward valve)**	Makes 30 seeded cosmetics real and gives every earned point a purpose. Costs nothing but a column and a shop page.
3	**Move to PostgreSQL**	Do it while the tables are still worthless. Migrating after real money is in them is a different and far worse job.
4	**Publisher payouts (stream 01)**	Turns revenue_pcp from a counter into a reason to publish. Needs the ledger integrity that step 3 provides.
5	**Carys can be bought (stream 02, inward valve)**	The first real money. Requires a payment provider, and requires step 3 to have happened.
6	**Subscriptions, then tournaments**	Both are priced in a currency that only means something once steps 2 and 5 are live.